22.2 工具执行运行时

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~900k tokens，输出 ~85k tokens（本章合计）

上一节我们看了工具是如何定义、组装和过滤的。本节深入工具从被 LLM 调用到返回结果的完整生命周期。

22.2.1 工具调用生命周期

完整调用链路

当 LLM 决定调用一个工具时，调用会经过以下步骤：

```
LLM 输出 tool_use → pi-agent-core 解析 → 工具分发
                                          │
                                          ┌──────────────────┐
                                          │  before_tool_call  │ ← 插件钩子（可拦截/修改）
                                          └──────────────────┘
                                          │
                                              ┌──────────┐
                                              │ 工具执行   │ ← abort signal 守卫
                                              └──────────┘
                                          │
                                          ┌──────────────────┐
                                          │  after_tool_call   │ ← 插件钩子（通知）
                                          └──────────────────┘
                                          │
                                          ┌──────────────────┐
                                          │ 结果格式化 + 持久化 │
                                          └──────────────────┘
                                          │
                                          ┌──────────────────┐
                                          │ tool_result_persist │ ← 插件钩子（可修改结果）
                                          └──────────────────┘
                                          │
                                            返回给 LLM
```

before_tool_call 钩子包装

每个工具在创建后都会被 `wrapToolWithBeforeToolCallHook()` 包装，注入钩子调用逻辑：

```ts
// src/agents/pi-tools.before-tool-call.ts
export function wrapToolWithBeforeToolCallHook(
  tool: AnyAgentTool,
  ctx?: { agentId?: string; sessionKey?: string },
): AnyAgentTool {
  const execute = tool.execute;
  return {
    ...tool,
    execute: async (toolCallId, params, signal, onUpdate) => {
      // 1. 运行 before_tool_call 钩子
      const outcome = await runBeforeToolCallHook({
        toolName: tool.name,
        params,
        toolCallId,
        ctx,
      });

      // 2. 如果被钩子阻止，抛出错误
      if (outcome.blocked) {
        throw new Error(outcome.reason);
      }

      // 3. 使用可能被钩子修改过的参数执行工具
      return await execute(toolCallId, outcome.params, signal, onUpdate);
    },
  };
}
```

`runBeforeToolCallHook()` 的内部逻辑：

```
export async function runBeforeToolCallHook(args) {
  const hookRunner = getGlobalHookRunner();

  // 快速路径：没有注册钩子时直接返回
  if (!hookRunner?.hasHooks("before_tool_call")) {
    return { blocked: false, params: args.params };
  }

  const hookResult = await hookRunner.runBeforeToolCall(
    { toolName, params: normalizedParams },
    { toolName, agentId, sessionKey },
  );

  // 钩子可以阻止调用
  if (hookResult?.block) {
    return { blocked: true, reason: hookResult.blockReason || "Blocked by plugin" };
  }

  // 钩子可以修改参数（浅合并）
  if (hookResult?.params && isPlainObject(hookResult.params)) {
    return { blocked: false, params: { ...params, ...hookResult.params } };
  }

  return { blocked: false, params };
}
```

Abort Signal 守卫

工具还会被 `wrapToolWithAbortSignal()` 包装，支持会话中断时取消长时间运行的工具：

```
// src/agents/pi-tools.abort.ts（简化）
export function wrapToolWithAbortSignal(
  tool: AnyAgentTool,
  signal?: AbortSignal,
): AnyAgentTool {
  return {
    ...tool,
    execute: async (toolCallId, params, existingSignal, onUpdate) => {
      // 将会话级 abort signal 与工具级 signal 组合
      return await tool.execute(toolCallId, params, signal, onUpdate);
    },
  };
}
```

包装顺序

工具的包装从内到外依次叠加：

```
原始工具 → 参数归一化 → Schema 归一化 → before_tool_call 钩子 → abort signal
```

在 `createOpenClawCodingTools()` 中可以清楚看到这个链：

```
// src/agents/pi-tools.ts（末尾部分）
const normalized = subagentFiltered.map(normalizeToolParameters);    // Schema 归一化
const withHooks = normalized.map((tool) =>                           // 钩子包装
  wrapToolWithBeforeToolCallHook(tool, { agentId, sessionKey })
);
const withAbort = options?.abortSignal                               // Abort 包装
  ? withHooks.map((tool) => wrapToolWithAbortSignal(tool, options.abortSignal))
  : withHooks;
return withAbort;
```

22.2.2 工具结果格式化

AgentToolResult 结构

工具执行后返回 `AgentToolResult`，核心结构如下：

```
type AgentToolResult<TResult> = {
  content: Array<
    | { type: "text"; text: string }
    | { type: "image"; data: string; mimeType: string }
  >;
  details?: TResult;
};
```

`content` 是发送给 LLM 的内容块数组，支持文本和图片两种类型。 `details` 是结构化的执行详情，不发送给 LLM，但可以用于日志和调试。

JSON 结果模式

大多数工具用 `jsonResult()` 返回结构化数据：

```
// 示例：cron 工具的返回
return jsonResult({
  ok: true,
  id: "cron_abc123",
  schedule: "0 9 * * 1",
  nextRun: "2025-01-06T09:00:00Z",
  status: "created",
});

// jsonResult() 将其序列化为带缩进的 JSON 文本
// → content: [{ type: "text", text: "{\n  \"ok\": true,\n  ..." }]
```

图片结果模式

`browser` 、 `image` 、 `canvas` 等视觉工具用 `imageResult()` 返回图片：

```
// 示例：browser 工具截图
return await imageResult({
  label: "browser",
  path: screenshotPath,
  base64: screenshotBuffer.toString("base64"),
  mimeType: "image/png",
  extraText: `Browser screenshot of ${url}`,
});
```

图片结果会经过 `sanitizeToolResultImages()` 处理，确保图片不会超过 LLM 的上下文限制。

工具结果持久化钩子

在工具结果写入会话转录之前，会触发 `tool_result_persist` 钩子（回顾 13.2.3 节），允许插件修改持久化的内容。典型应用场景：

- **精简大型结果**：移除工具结果中的冗余数据，减少上下文占用
- **脱敏处理**：在持久化前移除敏感信息
- **补充元数据**：添加时间戳或标签

22.2.3 错误处理与重试策略

工具级错误处理

工具执行中的异常会被 pi-agent-core 捕获，转换为工具错误结果返回给 LLM：

```
// pi-agent-core 内部（概念）
try {
  const result = await tool.execute(toolCallId, params, signal, onUpdate);
  return { type: "tool_result", content: result.content };
} catch (err) {
  return { type: "tool_result", is_error: true,
           content: [{ type: "text", text: String(err) }] };
}
```

LLM 看到错误结果后，通常会：

1. 分析错误原因
2. 修正参数后重试
3. 如果多次失败则放弃并告知用户

钩子错误隔离

`before_tool_call` 钩子的错误不会阻止工具执行——钩子被设计为"尽力而为"：

```
// pi-tools.before-tool-call.ts
try {
  const hookResult = await hookRunner.runBeforeToolCall(...);
  // 处理结果
} catch (err) {
  // 钩子失败只记录警告，不阻止工具执行
  log.warn(`before_tool_call hook failed: ${String(err)}`);
}
return { blocked: false, params };  // 继续执行
```

超时控制

`exec` 工具（命令执行）有独立的超时配置：

```
// 配置
tools:
  exec:
    timeoutSec: 120            # 单次执行超时
    backgroundMs: 30000        # 后台命令最大等待
    approvalRunningNoticeMs: 5000  # 等待审批时的提示延迟
```

工具执行还受到会话级 `AbortSignal` 的控制——当用户发送 `/stop` 命令或会话超时时，所有正在运行的工具都会收到中断信号。

本节小结

1. **工具调用经过四层处理**： `before_tool_call` 钩子拦截/修改 → 工具执行 → `after_tool_call` 通知 → `tool_result_persist` 持久化控制。
2. **工具被三层包装器链式包装**：Schema 归一化 → 钩子注入 → Abort Signal 守卫，保证了兼容性、可扩展性和可中断性。
3. **工具结果支持文本和图片两种格式**， `jsonResult()` 和 `imageResult()` 分别用于结构化数据和视觉内容。图片结果会经过 sanitize 处理。
4. **错误处理遵循"优雅降级"原则**：钩子失败不阻止执行，工具异常被转为错误结果让 LLM 自行修正，会话中断通过 AbortSignal 传播。